

▣ Les 01: Blockchain Fundamenten, EVM & Je Eerste Smart Contract

(Duur: 20 à 25 minuten • Vanaf scratch naar je eerste contract)

Welkom bij Les 01! Vandaag leggen we in 20 minuten het fundament van Ethereum en schrijven we je allereerste werkende smart contract.

▣ 1. Wat is de Ethereum Virtual Machine (EVM)?

Zie Ethereum niet als een munt, maar als een **wereldwijde, gedecentraliseerde computer**.

- **State Machine:** Ethereum heeft één wereldwijde “staat” (state): wie bezit welke tokens en wat staat er in het geheugen van elk smart contract?
 - **Smart Contract:** Een programmaatje dat permanent leeft op een specifiek blockchain-adres. Zodra het gedeployed is, kan niemand de code meer wijzigen (het is *immutable*).
 - **Gas:** Omdat duizenden computers (nodes) jouw code tegelijk verifiëren, kost elke berekening en opslagactie een kleine hoeveelheid “gas” (betaald in ETH). Dit voorkomt oneindige loops en overbelasting.
-

▣ 2. Anatomie van een Solidity Contract

Bekijk onderstaand contract. Dit is het universele *Hello World* van Web3: een contract om een getal op te slaan en uit te lezen.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/**
 * @title SimpleStorage
 * @dev Bewaart een getal op de Ethereum blockchain
 */
contract SimpleStorage {
    // 1. STATE VARIABLE: Wordt permanent opgeslagen in blockchain storage!
    uint256 public myFavoriteNumber;

    // 2. SCHRIJFFUNCTIE: Wijzigt de state van de blockchain (kost GAS)
    function setNumber(uint256 _newNumber) public {
        myFavoriteNumber = _newNumber;
    }

    // 3. LEESFUNCTIE (view): Leest alleen data, wijzigt niets (GRATIS om aan te roepen!)
    function getNumber() public view returns (uint256) {
        return myFavoriteNumber;
    }
}
```

De Belangrijkste Regels uit dit Voorbeeld:

1. `// SPDX-License-Identifier: MIT`: Verplichte open-source licentie-aanduiding bovenaan elk bestand.
 2. `pragma solidity ^0.8.20`;: Vertelt de compiler welke versie gebruikt moet worden. Vanaf versie `0.8.0` ben je automatisch beschermd tegen overflow/underflow!
 3. `uint256`: Een *unsigned integer* van 256 bits (een positief geheel getal van 0 tot $2^{256} - 1$).
 4. `public`: Maakt de variabele of functie bereikbaar van buitenaf. Solidity genereert voor `public` variabelen automatisch een gratis getter-functie!
 5. `view`: Geeft aan dat de functie **alleen leest** uit de blockchain. Het kost de gebruiker **geen gas** om een `view` functie aan te roepen vanaf een wallet!
-

□ 3. Oefening (iPad / Schrijf- & Denkopdracht)

Neem je Apple Pencil of open Remix IDE (remix.ethereum.org) en beantwoord/schrijf:

Vraag A: Begrip

1. Waarom kost `setNumber()` wel gas, maar `getNumber()` géén gas als je het via je wallet aanroept? _____
2. Wat betekent het dat een smart contract *immutable* is? _____

Opdracht B: Code Schrijven (Breid het contract uit!)

Schrijf hieronder een nieuwe functie `incrementNumber()` die `myFavoriteNumber` met 1 verhoogt:

```
// Vul de functie aan:  
function incrementNumber() public {  
    // Jouw code hier:
```

```
}
```

\vspace{0.8cm}

□ 4. Antwoordsleutel

Vraag A: 1. `setNumber()` wijzigt de toestand (state) van de blockchain (een schrijfoperatie). Alle nodes ter wereld moeten hun ledger bijwerken, wat computing power kost (gas). `getNumber()` is gemarkeerd met `view` en leest alleen lokaal uit een node zonder iets te veranderen. 2. *Immutable* betekent dat de code na deployment op het mainnet nooit meer aangepast of overschreven kan worden door wie dan ook.

Opdracht B (Uitbreiding):

```
function incrementNumber() public {  
    myFavoriteNumber += 1; // of: myFavoriteNumber = myFavoriteNumber + 1;  
}
```

□ **Huiswerk voor vandaag (10 minuten):**

1. Ga naar remix.ethereum.org op je Mac of iPad.
2. Maak een nieuw bestand `SimpleStorage.sol`, plak de code erin, klik op **Compile** en daarna op **Deploy (Remix VM)**.
3. Druk op `setNumber(42)` en klik daarna op `myFavoriteNumber` om te zien hoe de blockchain direct reageert!